

Implementation of a Transformer Layer in NumPy

1. Introduction

The goal of this assignment is to implement a single transformer layer using only NumPy, computing the forward pass. As required, I did not use PyTorch, did not implement back-propagation, did not consider a batch dimension, and left out the LayerNorm. All weights and biases are generated randomly. The layer has two parts: a multi-head dot-product self-attention block and a parallel feed-forward network, each followed by a residual connection. I tested it on three input sequences of 10, 50 and 100 tokens, where every token is a 256-dimensional embedding.

2. Inputs and Notation

Each input sequence is stored as a matrix X of shape $(\text{embedding_dim}, \text{token_size})$, so the columns are the token embeddings. I used $\text{embedding_dim} = 256$ for the embedding size, $\text{num_heads} = 8$ attention heads, $\text{dim_qkv} = \text{embedding_dim} // \text{num_heads} = 32$ for the per-head dimension of queries, keys and values, and $\text{dim_feed_forward} = 512$ for the hidden size of the feed-forward network. The three inputs are generated with `np.random.randn(embedding_dim, token_size)`.

3. Multi-Head Self-Attention

For a single head, the function `single_head_self_attention` computes the queries, keys and values by a linear transformation of the input, for example $\text{Query} = \omega_{\text{query}} @ X + \beta_{\text{query}}$. The attention scores are the scaled dot products $(\text{Key.T} @ \text{Query}) / \sqrt{\text{dim_qkv}}$. I divide by the square root of dim_qkv so the values entering the softmax do not become too large, which would push it into a region of very small gradients. The scores go through `softmax_columns`, which applies the softmax on each column subtracting the column maximum for stability, so for every query the weights are positive and sum to one. The head output is the weighted sum of the value vectors.

The function `multi_head_self_attention` runs the num_heads heads, stacks their outputs with `np.vstack`, and applies a final projection ω_c to combine them back to dimension embedding_dim . Since each head produces $\text{dim_qkv} = \text{embedding_dim} // \text{num_heads}$ rows, the concatenation has exactly embedding_dim rows, so the output matches the input shape and the residual connection works.

4. Parallel Feed-Forward Network

After the attention block, the function `parallel_MLP` processes each token with the same small network with one hidden layer of $\text{dim_feed_forward} = 512$ units and a ReLU activation: $W_2 @ \text{relu}(W_1 @ X + b_1) + b_2$. It is

applied column by column, so it works on every token independently and does not mix information across tokens that mixing is done only by self-attention. Here W_1 has shape $(\text{dim_feed_forward}, \text{embedding_dim})$ and W_2

maps back to embedding_dim , so the output again has shape $(\text{embedding_dim}, \text{token_size})$, as needed for the second residual connection.

5. Full Transformer Layer

The function `transformer_layer` puts the two blocks together with the two residual connections, computing $Z = X + \text{multi_head_self_attention}(X)$ and then $\text{out} = Z + \text{parallel_MLP}(Z)$ with No LayerNorm. I initialize one set of parameters with initial multi-head parameters and initial MLP parameters and apply the same layer to all three sequences.

6. Results and Checks:

Input	Output	mean	std
token_size = 10	256 × 10	-0.290	5.581
token_size = 50	256 × 50	-0.193	4.702
token_size = 100	256 × 100	-0.220	4.471

I also added a few checks to confirm the operations are correct:

- the attention matrix A has all entries greater than or equal to zero and each column sums to one, so it is a valid distribution over the keys.
- the same parameters work for any sequence length (I tested token_size of 1, 5, 200 and 500), since the parameters depend only on embedding_dim , num_heads and dim_feed_forward and not on the number of tokens, the layer has 526,080 parameters in total.
- applying parallel MLP to the whole sequence gives the same result as applying it token by token (`np.allclose` returns True), confirming it is truly position wise;
- stacking four transformer layers keeps the shape $(256, \text{token_size})$ end-to-end for all three inputs.

7. Conclusion

The layer reproduces the architecture: multi-head scaled dot-product self-attention with eight heads, followed by a position wise feed-forward network, each inside a residual connection and without LayerNorm. The checks confirm that the attention weights are valid, that the feed-forward part does not mix tokens, and that the same parameters handle sequences of any length, which is the parameter sharing that makes transformers work on variable length text.